forced-logout-flight-recorder

Forced-logout flight recorder — operator guide

Revision: 1 **Last modified:** 2026-08-21T16:10:00Z **Status:** active
Item: BOB-121 **Audience:** the operator of this workstation

What this is

A per-minute recorder that writes down what your user session looked like, so that if you get thrown back to the login screen again you can find out *why* by reading one file instead of digging through `journalctl` for an hour.

It watches. It does not act. It cannot kill anything, restart anything, or touch the machine's power state.

The 30-second version

You got logged out. Run this:

```
scripts/flight-recorder/flight-recorder.sh report
```

It prints a verdict. There are five, and they are mutually exclusive:

Verdict	What it means	What to do
REBOOT recorded	The boot id changed. The machine restarted.	The teardown is explained. Look for why it rebooted, not for a logout bug.
SUSPEND records present	The kernel logged a suspend entry.	Triage as a power-state event first (CONST-033 operational note).
SESSION TEARDOWN of	The user manager came back as a new	This is the incident class. Read the

Verdict	What it means	What to do
the forced-logout class	instance, with **no** reboot and **no** suspend.	OOM attribution line — see below.
RECORDING GAP with no teardown evidence	The recorder simply was not scheduled then.	Not an incident. Host was off, cron was stopped, or the tick was uninstalled.
no session-teardown event recorded	Nothing happened in this window.	Nothing to do.

Reading the OOM attribution

When the verdict is **SESSION TEARDOWN**, the deciding line is the OOM record's cgroup attribution, because it separates three genuinely different causes:

- **`libpod-…`** — a container hit *its own* memory limit. Containment worked as designed. Your session was collateral only if the container was in it.
- **`user@1000.service` / `user-1000.slice`** — a user-slice OOM. The kernel killed your session's processes because the slice ran out. This is a resource problem: look at the `mem_avail_kb` and `psi_mem_*` ramp in the ticks leading up to it.
- **no OOM record at all** — nothing ran out of memory. Something *sent* the kill. This is the BOB-126 shape, and the recorder cannot tell you who sent it — escalate to the `system.slice` watchdog (below) for initiator attribution.

That last case is what the seven incidents actually were. Verified against the real incident log:

```
$ grep -icE 'oom-kill|Killed process|Out of memory' docs/qa/
BOB-120/journalctl_23-42_to_23-46.log
0
$ grep -cE 'user@1000.service: Main process exited, code=killed'
docs/qa/BOB-120/journalctl_23-42_to_23-46.log
1
```

No OOM. One SIGKILL. External sender.

Install / remove

No `sudo`, no `su`, no root. That is deliberate — it is why this layer exists.

```
scripts/flight-recorder/install.sh --dry-run   # see exactly what
        would change
scripts/flight-recorder/install.sh             # start recording
```

To stop it:

```
scripts/flight-recorder/uninstall.sh           # stop, KEEP
        everything recorded
scripts/flight-recorder/uninstall.sh --purge   # stop and delete
        the recordings
```

Both only touch their own block in your crontab and back up the previous version first. Anything else in your crontab is left alone.

Check on it any time:

```
scripts/flight-recorder/flight-recorder.sh status
```

Where your data is

`~/.local/state/boba-flight-recorder/` — on `/home`, which is btrfs, so it survives the session dying. The recorder **refuses to run** if you point it at `tmpfs`, because a record that evaporates in the event it was recording is worse than no record.

Disk use is bounded at roughly 16 MB total by rotation, whatever happens.

It records numbers from `/proc`, `/sys/fs/cgroup` and `systemctl show` — no process command lines — so unlike the `system.slice` watchdog it cannot capture credentials that happen to be sitting in someone's `argv`.

What it will and will not do for you

It will: bracket the dead window to within 60 seconds; tell you whether the machine rebooted, suspended, or was torn down; show you the memory / PSI / thread-count ramp in the minutes *before* it happened, which nothing captured after the fact can recover; and tell you honestly when it could not read something rather than reporting a zero.

It will not: stop the kill. It is a flight recorder — it makes the crash investigable, it does not prevent the crash.

It has not yet been through a real one. Everything about its survival across a genuine `user@1000.service` teardown is reasoned from the cgroup topology and the incident journals, and confirmed

only against a staged analogue on a disposable unit. The first real incident will be the first real test. That is stated plainly here rather than dressed up.

The other half — and it needs you

This recorder survives the kill *mechanism observed in all seven incidents* (a cascade scoped to `user@1000.service`; the cron tick runs in a session scope that is a sibling of that unit, not a child of it). It does **not** survive a sweep of the entire `user-1000.slice`.

The mechanism that survives *everything* is the root-owned watchdog in `scripts/system-slice-watchdog/`, which also captures the journal window and the process tree around an incident — the initiator attribution this recorder cannot give you. It is written, reviewed, and **not installed**:

```
$ systemctl show boba-user1000-watchdog.service -p LoadState
LoadState=not-found
```

It needs one `su` from you, once:

```
bash scripts/system-slice-watchdog/install.sh   # prints the
        commands; you run them
```

No agent will run that for you. Until you do, this recorder is the only thing watching — and it is watching from one layer further out than anything the project had before, which was nothing.